One maintainer. One open-source repo. Aligned with 23 security & privacy standards.

We did *not* write 23 checklists.

How good architecture — not a pile of paperwork — keeps a small open-source project audit-ready across the world's privacy and security rules, with no compliance team and no audit budget.

`↳ each technical claim names the file that backs it · a CI drift gate keeps the tool docs honest against the code`

The standards wall

23 frameworks · 6 domains · across the US, EU, UK & global bodies.

Domain	Where it applies	Frameworks aligned to
Information security	Global · US · UK	ISO 27001 · SOC 2 Type II · NIST CSF 2.0 · FedRAMP · BSIMM · UK Cyber Essentials · NCSC CAF
Privacy & data rights	EU · UK · US · APAC	GDPR / UK GDPR · Global CBPR
Healthcare data	US	HIPAA · HITRUST CSF
AI & agentic systems	EU · US · Global	EU AI Act · NIST AI RMF · OWASP LLM Top 10 · OWASP Agentic Top 10
AI-tool (MCP) security	Global · US	OWASP MCP · CSA MCP · NSA MCP Guidance
Supply chain & protocol	EU · Global	EU CRA · NIS2 · FIRST PSIRT · MITRE ATT&CK · RFC 9700/9449

No compliance team. No budget for 23 audits. So how?

↳ proof: docs/SECURITY_AND_COMPLIANCE.md → "Standards Alignment"

The trick: 23 standards → 8 properties

Strip the labels, and they demand **the same handful of things**:

Access control

Encryption rest + transit

Audit trail

Data minimization

Tenant isolation

Supply-chain integrity

Vulnerability handling

Transparency

Compliance teams call this *control mapping* — and it's usually a 1,400-row spreadsheet and a consultant. **We built the 8 properties into the code instead.**

> **The payoff:** add the 23rd standard and you write zero new code — you already satisfy it. Compliance cost stops scaling with the number of regimes.

↳ proof: `docs/SECURITY_AND_COMPLIANCE.md "Compliance Posture"` · `docs/SECURITY.md crosswalks`

The architecture that satisfies them

The 8 properties are baked into *how the code is built* — so they hold everywhere, not just where someone remembered to add them:

- **One way in for every dependency** — nothing reaches in through a back door, so there's one place to audit
- **Swappable parts behind clean seams** — the cache, search, and audit layers can change without rewriting callers
- **One encryption routine** — AES-256 everywhere data is stored, never re-implemented per feature
- **One safe web-fetch client** — *every* outbound request goes through the same SSRF-checked path
- **An audit log on every tool call** — structured, secrets stripped out, and it never slows the request

> **"Compliance through architecture, not bolt-on checklists."**

↳ proof: CLAUDE.md "Design Rules" · internal/cache/crypto.go · internal/scraper/ssrf.go

The safest config is the *default* config

Run it the normal way — locally, inside your AI app — and there's **no network port open, no login to misconfigure, nothing exposed to the internet.** The app that launched it is the only thing that can reach it. The typical user *can't* hold it wrong, because there's nothing to set.

The heavyweight controls for shared, multi-user deployments — sign-in, per-customer isolation, rate limits, encryption, audit logs — only switch on when an operator deliberately runs it as a network server.

> **Secure by default, permissive by configuration. Power is unlocked by an *explicit* choice, never the reverse.**

↳ proof: docs/SECURITY_AND_COMPLIANCE.md Principles 1 & 4, "Deployment Security"

AI tools have brand-new threat classes

Give an AI a tool that fetches any web page for it, and you've created two attack surfaces ordinary web-security checklists miss:

It can be tricked into attacking your own network (*SSRF*). So before any fetch we check every resolved address against 19 blocked ranges + cloud credential endpoints, connect only to that exact address, and re-check on every redirect — a hijacked link can't reach inside.

The pages it reads are untrusted input — a booby-trapped page can try to hijack the AI (*prompt injection*). So scraped text is cleaned, size-capped, and clearly flagged to the model as "data, not instructions."

Why now: prompt injection is the **#1 risk** on OWASP's LLM list, and the major AI standards don't yet cover *agentic* tools like this one — making it a live case study for a gap regulators are still writing.

↳ proof: internal/scraper/ssrf.go · docs/SECURITY_AND_COMPLIANCE.md "SSRF" & "Content Security"

An erasure registry you can't outrun

Privacy starts with collecting little: the cache is content-addressed (not keyed to a user), and the operator — not us — is the data controller. But the moment a feature *does* store personal data, "right to be forgotten" has to actually work.

So it's enforced structurally: **every store that holds personal data must register an Exporter + Eraser.** One `(tenant, user)` request fans out to all of them — and each store ships a round-trip **release-gate test**.

> **Add a new feature and forget to wire its eraser? CI fails — not an auditor. GDPR access / portability / erasure (Art. 15 / 17 / 20) becomes a property of the build, not a promise in a policy PDF.**

shipped in v1.13

↳ proof: `internal/datasubject/registry.go` · `internal/session/datasubject_test.go` ("the #85 release gate")

Consent as a primitive: record → verify → honor

The AI-tool standard (MCP) says *asking* the user for consent is the **client app's** job — so most servers do nothing. But whoever *stores* the data is, in law, the **data controller**, and GDPR / Quebec Law 25 make *them* prove consent was given and honor a withdrawal — a duty a login token can't discharge.

So the server treats consent as three things it actually does:

- **Records** it — encrypted, logged, with a typed purpose ("memory," "analytics," "workspace").
- **Verifies** it on every access — no record, no processing (it defaults to *off*).
- **Honors** it — a withdrawal automatically erases the data it covered.

shipped in v1.13

↳ proof: `internal/consent/consent.go` · `internal/consent/store.go (HasConsent, fail-closed)`

The docs are tested, not trusted

"Keep the docs accurate" isn't a good intention here — it's enforced by a stack of small mechanisms, so drift is hard to write and impossible to merge quietly:

Layer	What keeps it honest
Code is the source of truth	Docs never hardcode tool lists, counts, versions, or env tables — they point at the file that defines them, so there's nothing to fall out of sync
Rules the AI writes by	The agent guide (`CLAUDE.md`) makes "every claim links to its file" and "no duplicated facts" mechanical rules for Claude / Copilot / Cursor — the way most code now lands
A test reads the docs	At build time it parses `docs/TOOLS.md` , starts a real server, and fails the build if the documented tools, output shapes, or read/write flags don't match reality
Gates that can't be skipped	A pre-commit hook catches it locally; CI re-runs the drift check on **every** PR — even docs-only ones, so a doc edit alone can't slip past

> **The result: the claims in these docs can't quietly stop being true.** *Read the code, not the marketing* **— because here, the docs answer to the code.**

↳ proof: CLAUDE.md "Documentation Guidelines" · internal/tools/metadata_test.go · .githooks · .github/workflows/ci.yml (docs-drift)

So what is this actually worth?

The same architecture pays off differently for everyone who touches it:

If you're...	What you get
A user	The safe setup is the *default* — nothing to configure, nothing to leak
An operator / buyer	One small binary you can take to a hospital, an EU regulator, or a federal review — evidence links to code, not slideware
A developer / contributor	Add a feature and the guardrails (consent, erasure, audit, tests) come *with* it — you can't ship the unsafe version
A founder / eng leader	Compliance cost scales with *features you turn on*, not with the number of regulations or headcount

> **Compliance stops being a project you fund and becomes a property you inherit.**

What transfers to any project

1. **Map standards to primitives, not checklists** — satisfy many at once.
2. **Make the default the safe one** — gate power behind explicit config.
3. **Let compliance scale with features** (tiers) — not a big-bang program.
4. **Encode the rules as constraints, enforce them in CI** — including for AI coding agents.
5. **If a doc can be wrong without a test failing, it will be** — so test it.

Honest boundaries: "aligned with," not "certified." The local and server threat models differ. By default it stores little; the features that do store personal data are opt-in, consent-gated, and erasable — not absent.

Read the code, not the marketing.

Each technical claim here names the file that backs it — open any one and check.
And a CI drift gate keeps the tool docs honest against the code.

The receipts: `v1.10` tenant isolation → `v1.11` zero-config fallback → `v1.12` HTTP hardening → `v1.13` consent + GDPR erasure.

*Solo maintainer · MIT licensed · **contributors welcome.** Spot a claim that doesn't match the code? Open an issue or a PR — that's the whole point. Come help build it:* `github.com/zoharbabin/web-researcher-mcp`